

★ WEEK 8 · PART 1 of 3

State Space Models

A Hidden State, Glimpsed Through Noise

The two equations · the local level model · the framework behind everything

Part II opens with one unifying language. Behind the Kalman filter, structural models, and time-varying parameters lies a single idea: a hidden state you can only observe through noisy data.

1

PART 1 · STATE SPACE

The Unifying Framework

The hidden-state idea, the two equations that define every state space model, the local level model, and how familiar models like AR(1) are just special cases.

Shooting at What You Cannot See — Part 1

How a wartime gunnery problem birthed filtering

In World War II, gunners had to hit aircraft they could only track through noisy, jumpy radar. The true trajectory was hidden; the measurements were unreliable. How do you aim at what you cannot cleanly see?

The problem

- The target's true position and velocity were unobserved.
- Radar gave noisy, error-ridden snapshots.
- You needed the best estimate of the hidden truth, in real time.

The breakthrough

- Separate the SIGNAL (true trajectory) from the NOISE.
- Norbert Wiener's filtering theory tackled exactly this.
- It reframed tracking as estimating a hidden state.

Shooting at What You Cannot See — Part 2

The state space idea, and the lesson

Rudolf Kalman turned this into a recursive state-space solution in 1960 — the Kalman filter. It guided Apollo to the Moon, and it is the same idea we now apply to markets.

What it became

- A model of a hidden state evolving over time.
- Updated efficiently as each noisy observation arrives.
- The backbone of navigation, control, and finance.

The lesson for us

- Markets, too, hide their true state behind noisy prices.
- Trend, volatility, and hedge ratios are hidden states.
- State space gives us one language to estimate them all.

From a Toolbox to One Language

Where we are as Part II begins

1

Part I built a toolbox

ARIMA, GARCH, VAR, cointegration — a set of specialized models, each strong for its own kind of structure.

2

Part II offers a unifying framework

State space models express nearly all of them in one common form. The same two equations, with different matrices, recover each special case.

3

And it unlocks new power

Hidden states, time-varying parameters, missing data, and real-time updating — things a fixed model cannot do. This is the language of the Kalman filter (Week 9).

Part 1 in one line: one framework — a hidden state seen through noise — contains them all.

The Core Idea

A hidden state, observed only through noise

A state space model separates what you WANT to know (a hidden state) from what you can SEE (noisy observations). The state evolves over time; each observation is a noisy glimpse of it.

The hidden state

- The quantity you actually care about — a level, a trend, a ratio.
- It evolves through time by its own rule.
- You never observe it directly.

The noisy observation

- The data you actually record.
- It is the state plus measurement error.
- Your job: infer the state from these noisy glimpses.

The Two Equations

Every state space model is built from these

$$\begin{aligned} \text{state:} \quad & x_t = T x_{t-1} + \eta_t \quad (\eta_t \sim \mathcal{N}(0, Q)) \\ \text{observation:} \quad & y_t = Z x_t + \varepsilon_t \quad (\varepsilon_t \sim \mathcal{N}(0, H)) \end{aligned}$$

The state equation

- Describes how the hidden state evolves each period.
- The matrix T is the transition — the state's own dynamics.
- A random shock keeps the state from being deterministic.

The observation equation

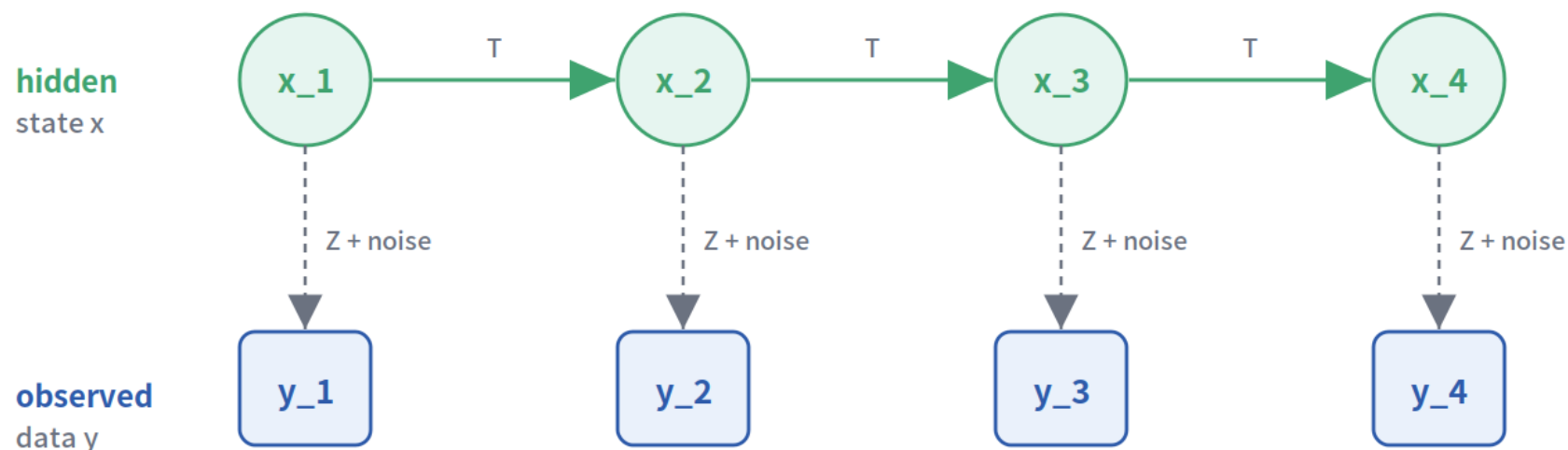
- Links the hidden state to the data you actually see.
- The matrix Z maps state to observation.
- Measurement noise separates the truth from the record.

The State Space Picture

A hidden chain of states evolves by the transition T ; each state emits a noisy observation through Z . We only ever see the bottom row — and infer the top from it.

The State Space Picture

A hidden chain of states evolves; we see only noisy snapshots of it.

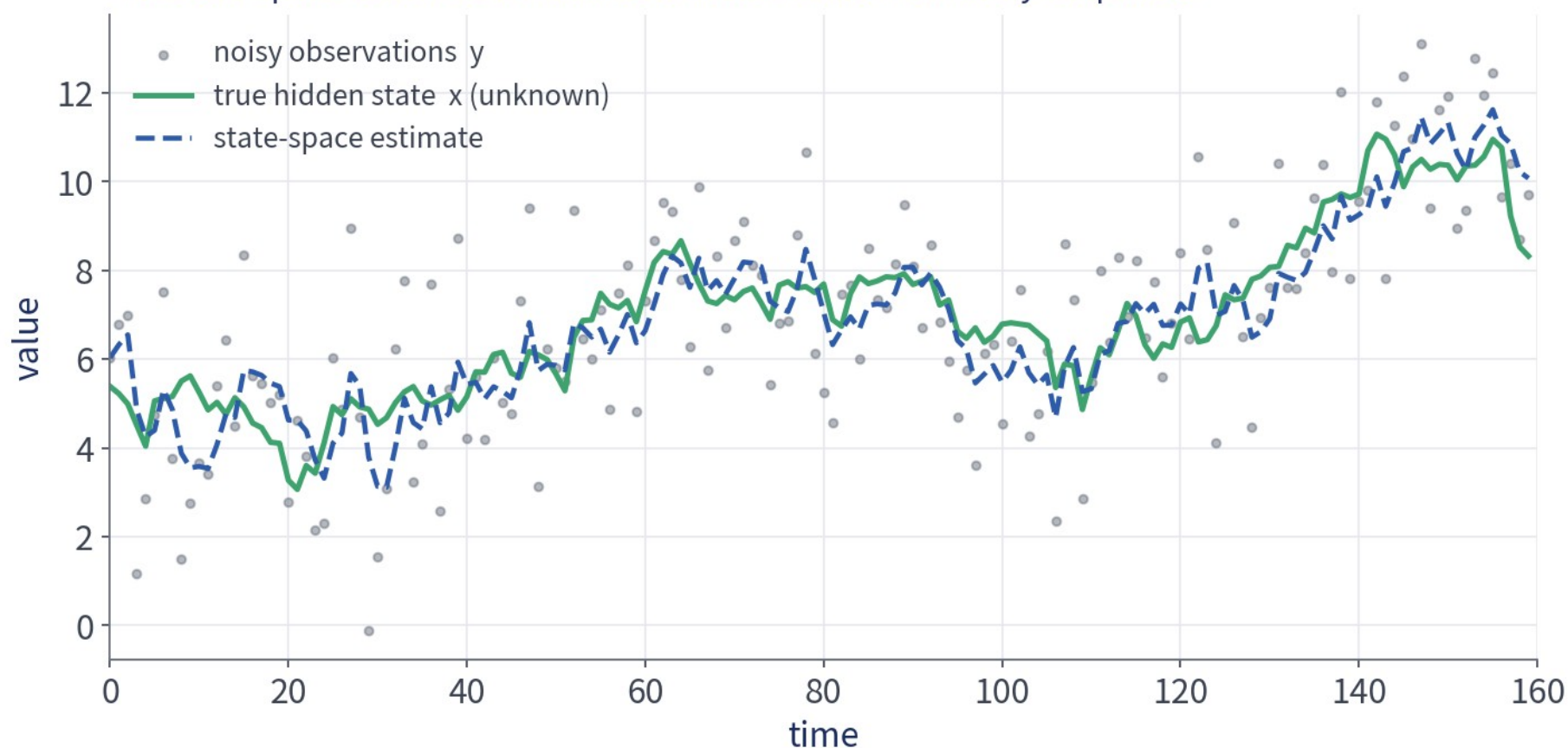


The state evolves by T (transition); each observation is Z times the state plus measurement noise. We infer x from the y 's.

Recovering the Hidden State

From a scatter of noisy observations, a state space model reconstructs a smooth estimate of the underlying hidden state — close to the true (unobserved) path.

The state-space idea: recover a smooth hidden state from noisy snapshots



The Local Level Model

The simplest state space model of all

$$\text{local level: } x_t = x_{t-1} + \eta_t, \quad y_t = x_t + \varepsilon_t$$

What it says

- The hidden level drifts as a random walk.
- Each observation is that level plus noise.
- It is 'a random walk seen through a fog'.

Why it matters

- It is the building block for everything richer.
- It already does something ARIMA cannot: extract a smooth level.
- Add a slope or a season and you get structural models.

Signal vs Noise

How much should you trust the latest data point?

signal-to-noise $q = \frac{\sigma_\eta^2}{\sigma_\varepsilon^2}$: large $q \Rightarrow$ trust data; small $q \Rightarrow$ smooth

The dial

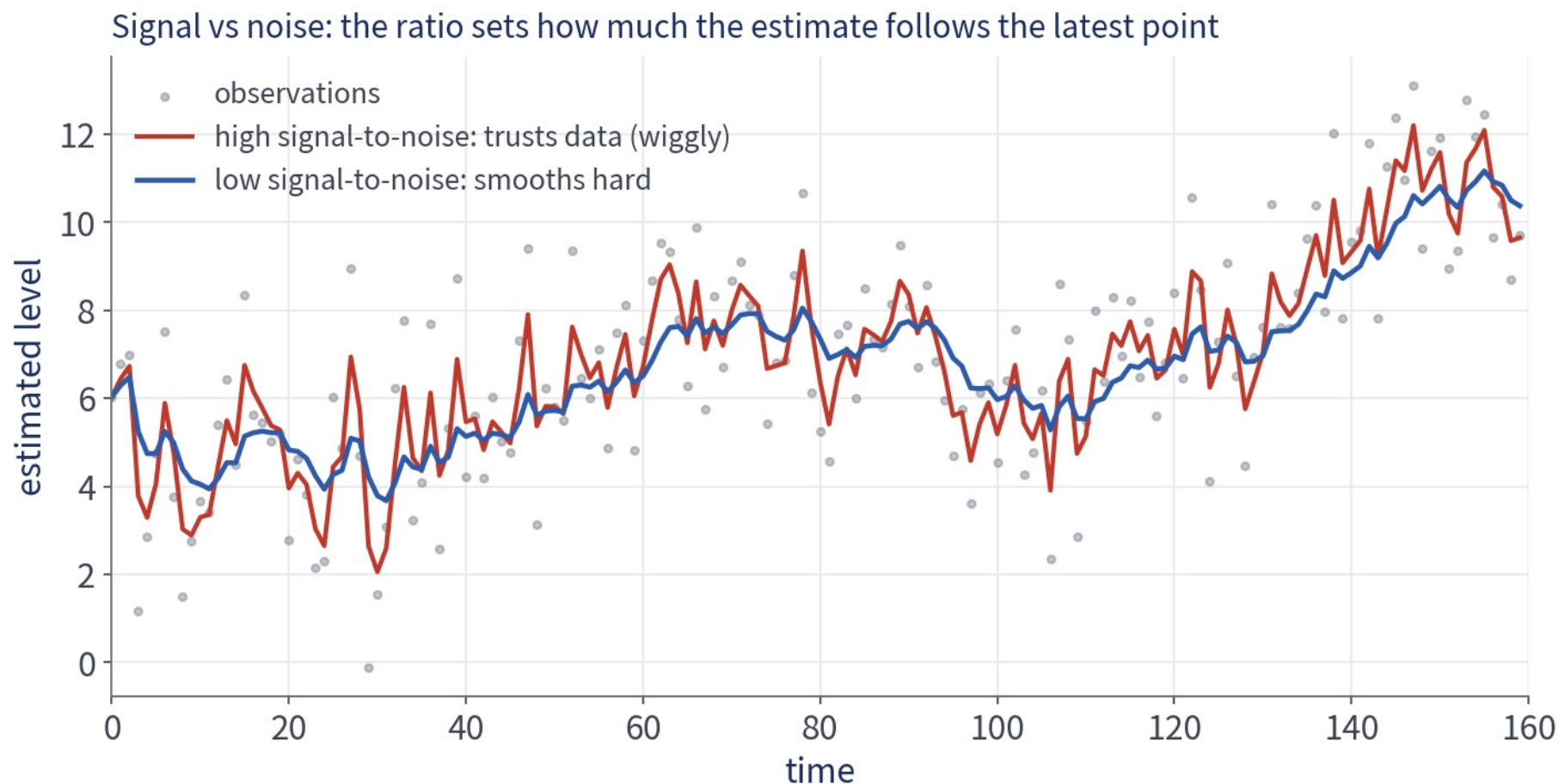
- The signal-to-noise ratio governs everything.
- High ratio: the state moves a lot — trust each new point.
- Low ratio: the state barely moves — smooth aggressively.

Why it is the key choice

- It decides how reactive vs stable your estimate is.
- It is estimated from the data, not set by hand.
- It is the same trade-off behind every filter (Week 9).

Trusting the Data, or Smoothing It

A high signal-to-noise ratio makes the estimate chase each observation (red); a low ratio smooths hard through the noise (blue). The ratio is the single most important choice.



ARIMA in Disguise: AR(1) as State Space

The familiar models are special cases

$$\text{AR}(1) \text{ as state space: } x_t = \phi x_{t-1} + \eta_t, \quad y_t = x_t$$

The mapping

- Let the transition be ϕ and the observation be exact.
- The state equation becomes an AR(1).
- No measurement noise — the state IS the series.

Why this is powerful

- ARIMA, ARMA, and more all fit this template.
- One estimation engine handles them all.
- You gain state space's extras for free — filtering, missing data.

One Framework Contains Them All

Why state space is the lingua franca of time series

Change the matrices T , Z , and the noise, and the same two equations become an AR, an ARIMA, a local trend, a structural model, or a time-varying regression. One form, many models.

What varies

- The transition matrix T sets the state's dynamics.
- The observation matrix Z sets how you see it.
- The noise variances set the signal-to-noise balance.

What stays the same

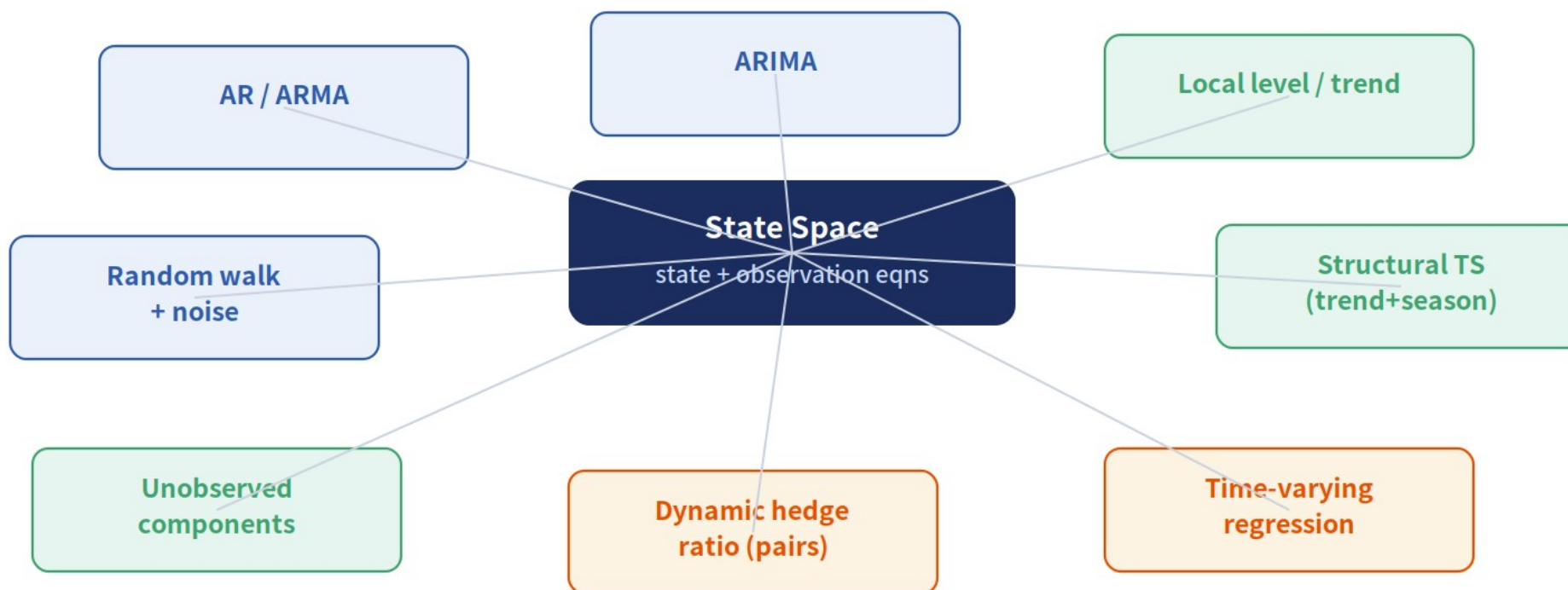
- The two-equation structure never changes.
- One estimation and inference engine covers all cases.
- Learn it once; apply it everywhere.

State Space as the Hub

AR/ARMA, ARIMA, local level and trend, structural models, time-varying regressions, dynamic hedge ratios — all are the same two equations with different matrices.

State Space as the Hub

Same two equations, different matrices - one framework contains them all.



Four Things State Space Buys You

Beyond what a fixed model can do

Capability	What it means
Hidden components	Extract an unobserved level, trend, or cycle directly.
Time-varying parameters	Let coefficients drift as the world changes.
Missing / irregular data	Handle gaps and uneven sampling naturally.
Real-time updating	Revise estimates the instant new data arrives.

Each of these is awkward or impossible in a fixed ARIMA — and routine in state space.

Time-Varying Parameters

Let the coefficients themselves drift

In a fixed model, a coefficient is one number for all time. In state space, a coefficient can BE a hidden state — free to evolve — capturing relationships that change with the regime.

The idea

- Treat a parameter as a state that follows its own random walk.
- The filter estimates its value at every point in time.
- The model adapts instead of assuming constancy.

Why it matters in finance

- Betas, hedge ratios, and risk exposures all drift.
- A fixed estimate is stale the moment the regime shifts.
- This sets up the dynamic hedge ratio (Part 2).

Missing and Irregular Data

Gaps are no obstacle

How it copes

- When an observation is missing, just skip the update step.
- The state still evolves; you simply don't correct it that period.
- Irregular timing is handled the same graceful way.

Why this is a big deal

- Real financial data has holidays, halts, and gaps.
- ARIMA needs a complete, evenly-spaced series.
- State space works around the holes automatically.

The Local Linear Trend Model

Add a slope state, and the model tracks trends

$$\text{local trend: } \mu_t = \mu_{t-1} + \beta_{t-1} + \eta_t, \quad \beta_t = \beta_{t-1} + \zeta_t, \quad y_t = \mu_t + \varepsilon_t$$

What's added

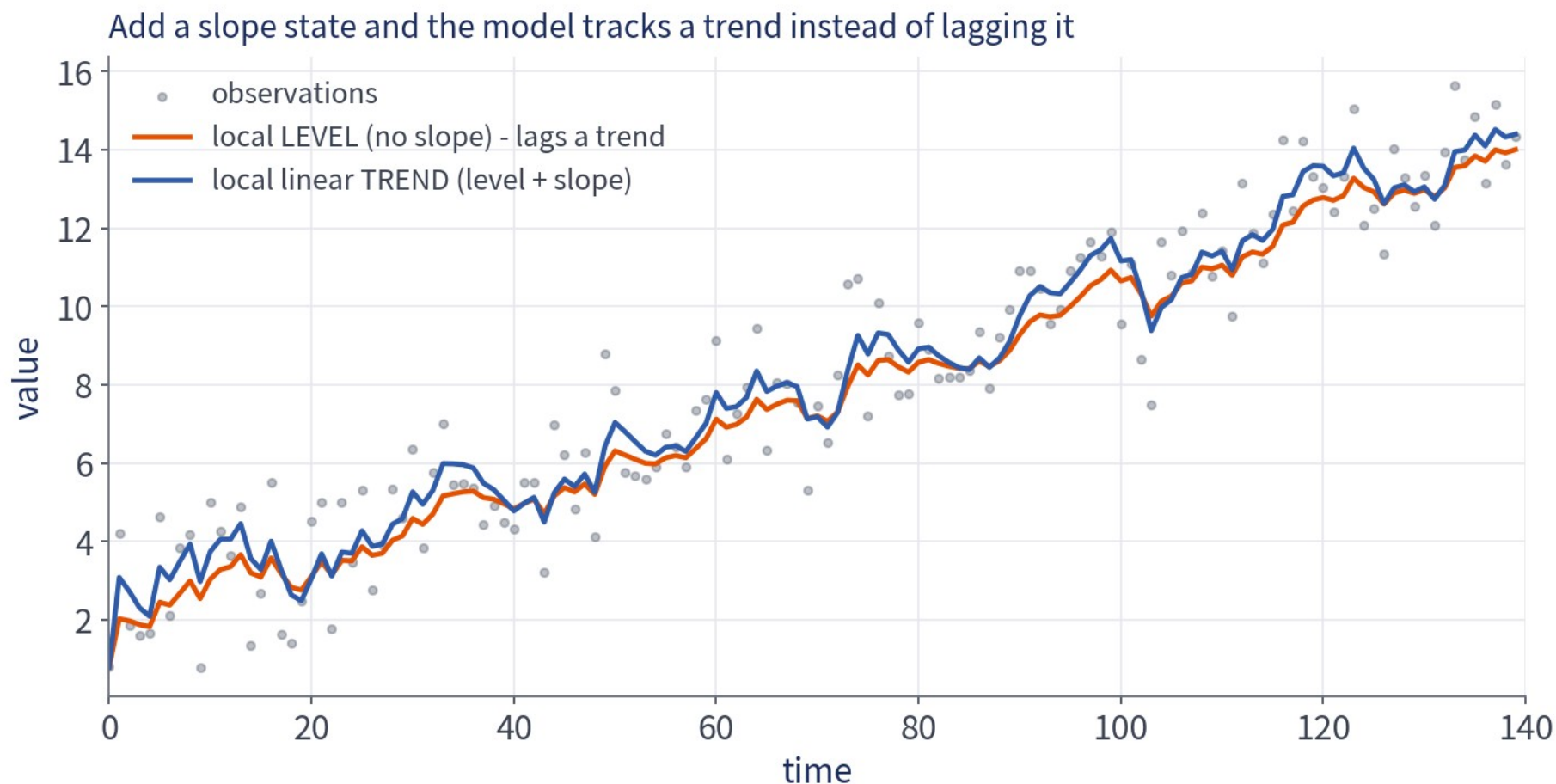
- A second state — the slope — evolves alongside the level.
- The level now moves by its own drift plus the current slope.
- Both are estimated in real time.

What it buys

- The model follows a trend instead of lagging behind it.
- It adapts as the slope itself changes.
- It is the workhorse of structural time-series decomposition.

Level vs Local Linear Trend

A local level alone lags a trending series (orange); adding a slope state lets the model track the trend as it moves (blue). Extra states buy extra structure.



Structural Time Series

Trend, season, and cycle — as states

The idea

- Give each feature its own state: level, slope, season, cycle.
- Stack them, and the model decomposes the series.
- Each component is estimated and can be read off directly.

Why it is appealing

- You SEE the trend and season, not just a forecast.
- Each part evolves and adapts over time.
- It is interpretable in a way ARIMA rarely is.

The State Space Workflow

How you actually use it in practice

1 · Specify

the states

Decide the hidden states: level, slope, season, or a parameter.

2 · Write eqns

T, Z, noise

Cast them in the two-equation form.

3 · Estimate

variances

Fit the noise variances by maximum likelihood.

4 · Filter

infer states

Run the filter to estimate the states over time.

5 · Use

read+forecast

Read components, forecast, and update live.

Steps 4–5 — the filtering that estimates the states — are the subject of Part 2 and Week 9.

Fit a Local Level Model

Direct Claude Code to extract a hidden level

GOAL Fit the simplest state space model to a noisy series, extract the hidden level, and read off the signal-to-noise balance.

[INPUT] Prompt · Claude Code

Write and run `local_level_lab.py` that:

1. loads a noisy series (e.g. a short window of daily prices, or a noisy macro series);
2. fits a local level model with `statsmodels.UnobservedComponents('local level');`
3. plots the data with the filtered and smoothed level estimates overlaid;
4. reports the estimated state and observation variances and the implied signal-to-noise.

Tell me whether the series is closer to a random walk or to pure noise around a level.

How to work with it

- Filtered uses data up to now; smoothed uses all data.
- A high signal-to-noise $\rightarrow$ the level tracks the data.
- A low ratio $\rightarrow$ a nearly flat, smooth level.
- Keep it; Part 2 explains filter vs smooth.

Deliverable: `local_level_lab.py` + a data-and-level plot with the variances.

Three Questions Before You Move On

If you can answer these, Part 1 has landed

Q1	What are the two equations that define every state space model?	State + observation
Q2	What does the signal-to-noise ratio control?	How much to trust new data
Q3	How can an AR(1) be written as a state space model?	Transition ϕ, exact obs

All three come from the two equations, the noise balance, and the unifying view.

Three Misconceptions to Avoid

Where state space intuition slips

Common myths

- "State space is a brand-new model to memorize."
- "The hidden state is just a smoothed version of the data."
- "You must set the signal-to-noise ratio by hand."

The reality

- It is a FRAMEWORK — your old models are special cases of it.
- The state has its own dynamics; smoothing is one output, not the idea.
- The noise variances are estimated from the data by likelihood.

Three Things to Carry Forward

Part 1 in one screen

1

A hidden state, seen through noise

Every state space model has two equations: how a hidden state evolves, and how noisy observations reveal it. The local level model is the simplest example.

2

One framework contains them all

Change the matrices and the same two equations become AR, ARIMA, local trend, or structural models — with hidden components and time-varying parameters as a bonus.

3

The signal-to-noise ratio is the key dial

It decides how much to trust each new observation versus smooth through it — the same trade-off that the Kalman filter will resolve optimally.

Next — Part 2: filtering, smoothing, and forecasting, and a drifting hedge ratio for pairs.